

UML DIAGRAM

- ① WHAT IS UML DIAGRAM
- ② TYPES OF UML DIAGRAM

NOTES IN GITHUB
GITHUB LINK IN DESC.

① METHODS OF COMMUNICATIONS

- ① Using Words
 - Email
 - Meetings
 - Chat

Cons

- ① Missing Visual context
- ② Miss communication
- ③ Multiple Interpretations
(Ambiguity)

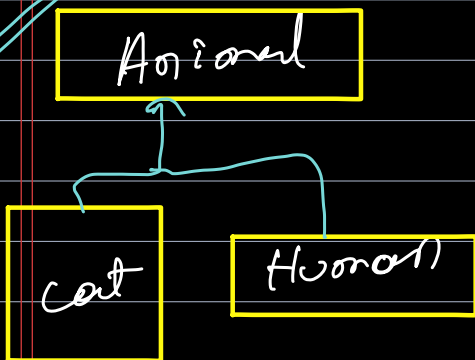
② Visuals / Pictures / Diagrams

- ① No need to visualize
- ② less scope for ambiguity

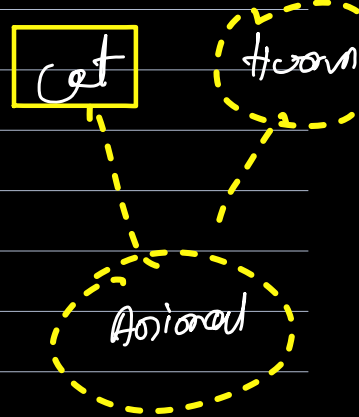
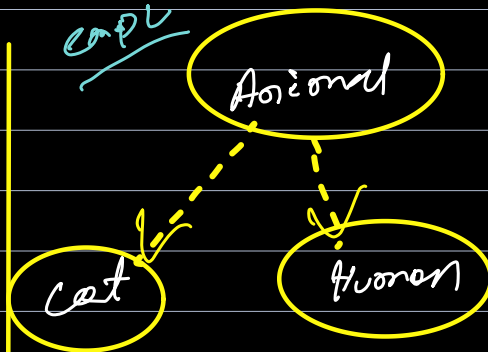
Cons

- ① No uniformity / standardization

expt



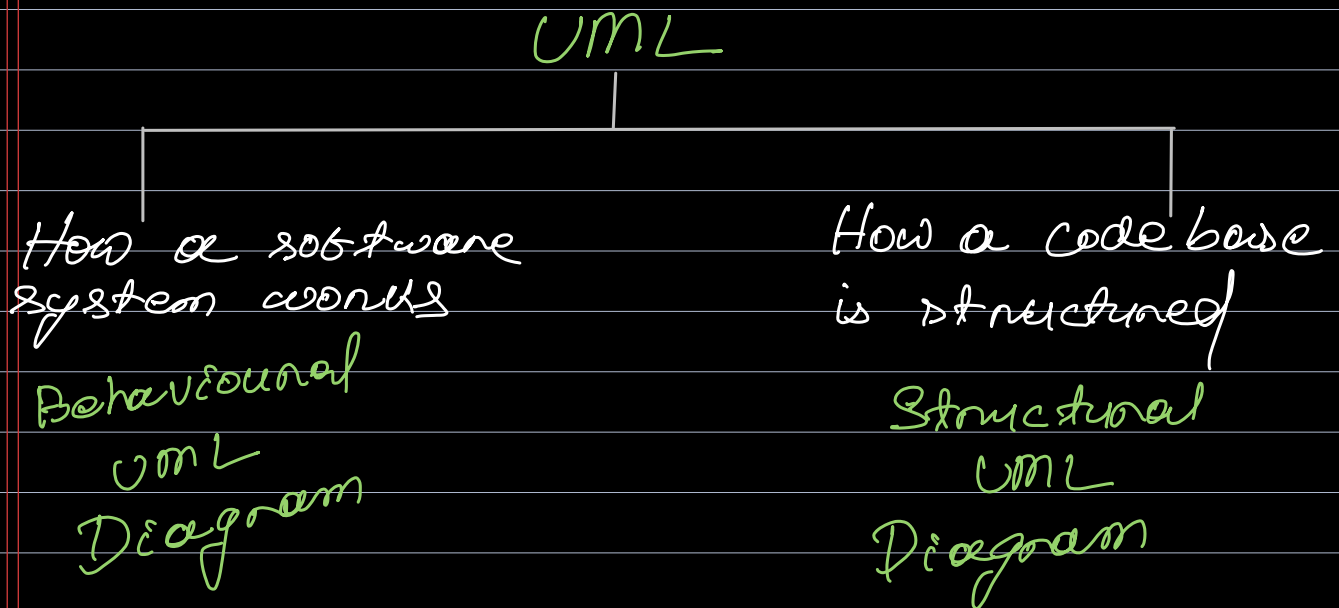
expt



There should be standardization in diagram to ensure proper understanding across software industries.

Unified Modeling Language (UML)

② TYPES OF UML DIAGRAM



Behavioral Diagram

- ① Use Case Diagram
 - ② Sequence Diagram
 - ③ Activity Diagram
- etc.

STRUCTURE DIAGRAMS

- ① Class Diagram
- ② Component Diagram
- ③ Package Diagram
- ④ Object Diagram etc.

UML USE CASE DIAGRAM

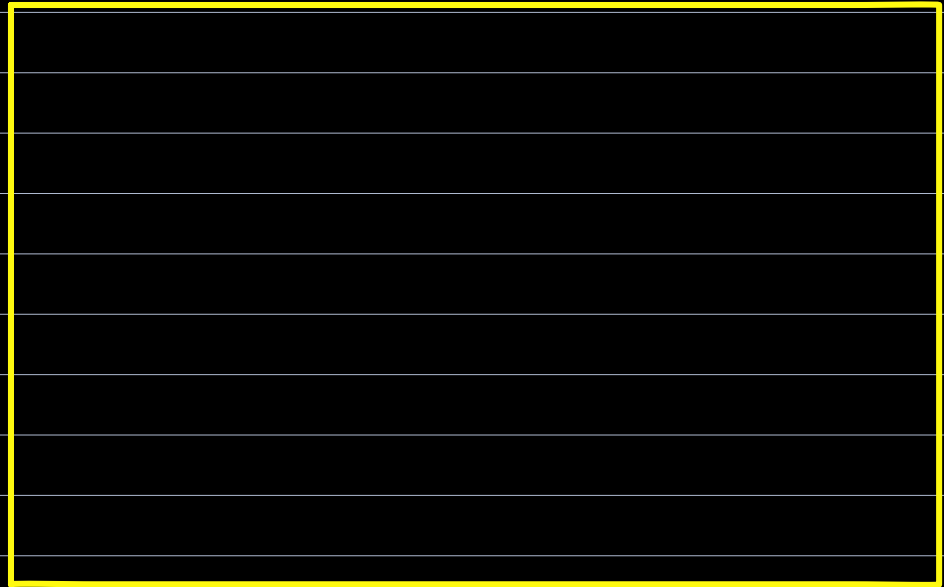
- ① Definition
- ② How to represent
 - System Boundary
 - Use case
 - Actor
- ③ Relationships
- ④ Draw E-commerce

Definition

- ① Comes under behavioural UML diagram
- ② It can summarize the details of system's users (Actors) and their interaction with the system.
- ③ People often apply use case modeling for the following purpose.
 - ① Specify the content of a system.
 - ② Represent the goals of system-user interactions.
 - ③ Defines functional requirement in a system.

How to represent System Boundary

- ① Create a boundary between what is supported by the system and what is not supported by the system.
- ② Every function that is supported by system will go inside boundary.
- ③ we can represent this using



How to represent Usecase

① Function / Feature / Functionality that is supported by any S/W system will be represented over inside which you put the name of the feature.

② Representation :

Ex:

login

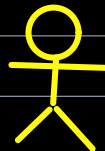
Signup

view Dashboard

How to represent Actor

① Someone who can use diff use case of a system.

② Representation :



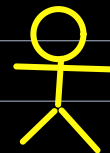
Ex:



User



Employee



Payment gateway

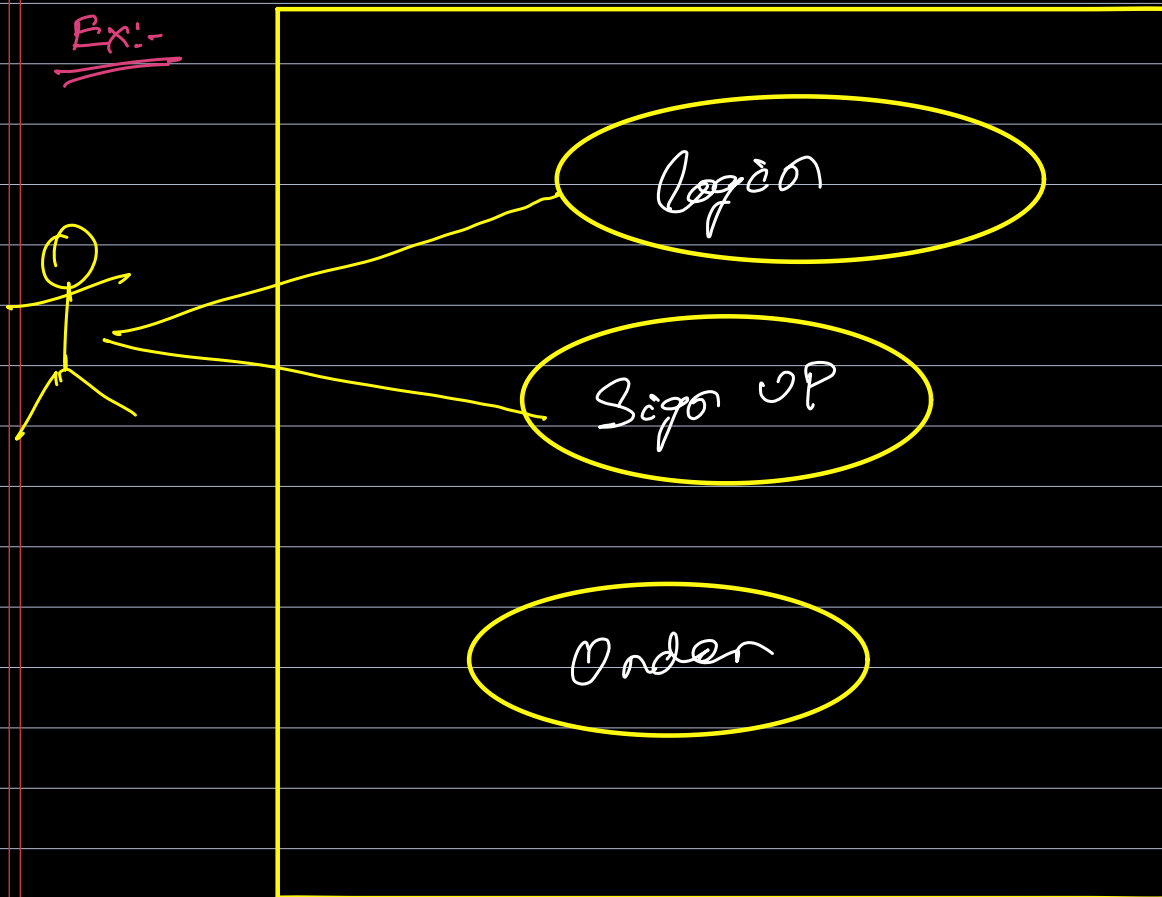
Relationships

- ① Association
- ② Include
- ③ Extend
- ④ Generalization

Association

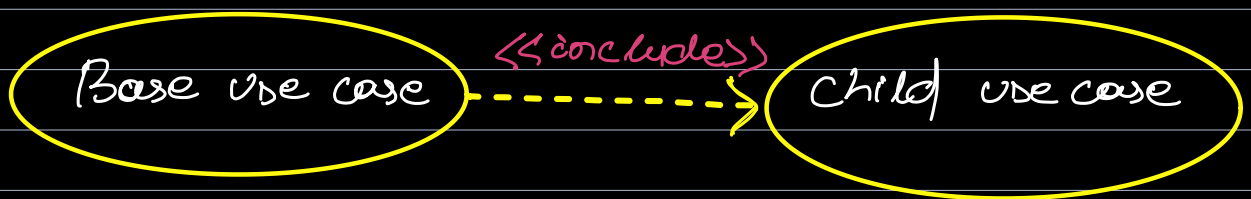
- ① It represents a communication path between an Actor and a use case.
- ② Representation : _____

Ex:-

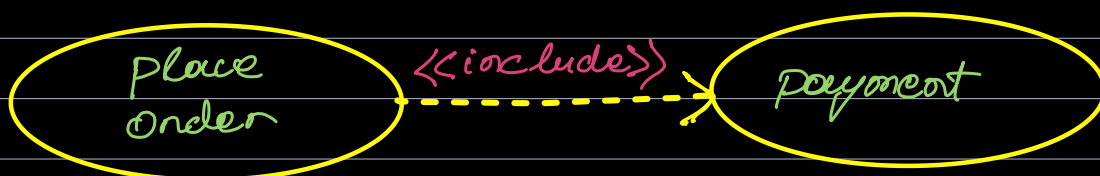
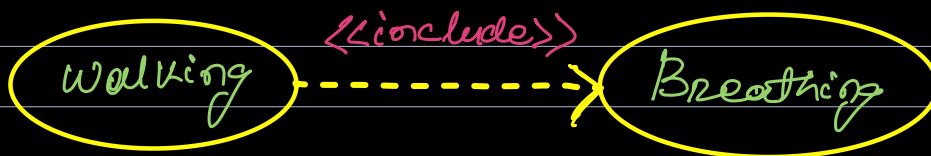
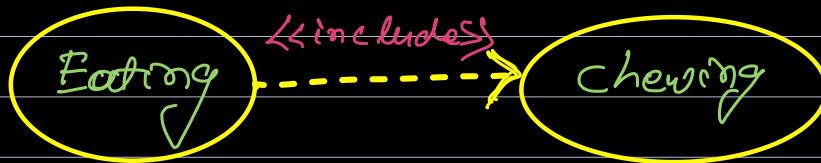


Include

- ① One use case must use another use case to complete it's task.
- ② Mandatory step
- ③ Representation:- 



Ex

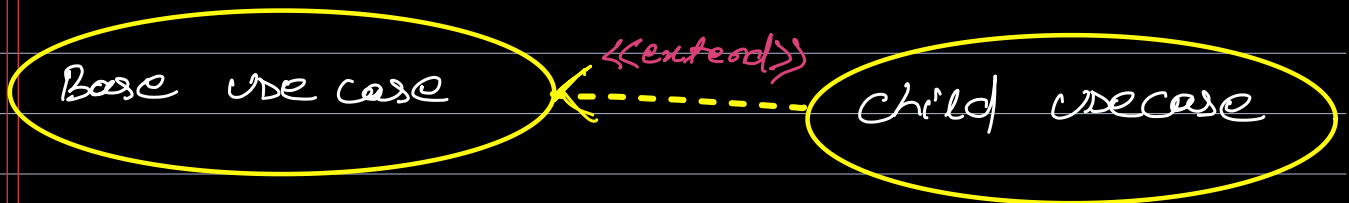


Extend

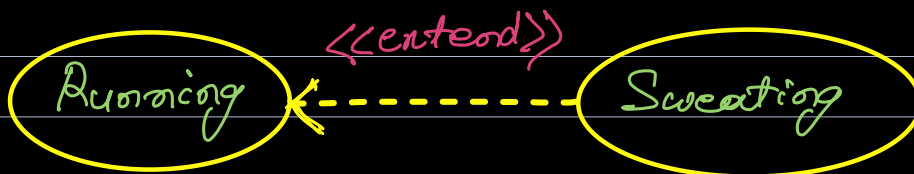
① Used for optional behaviour that only happens under specific conditions.

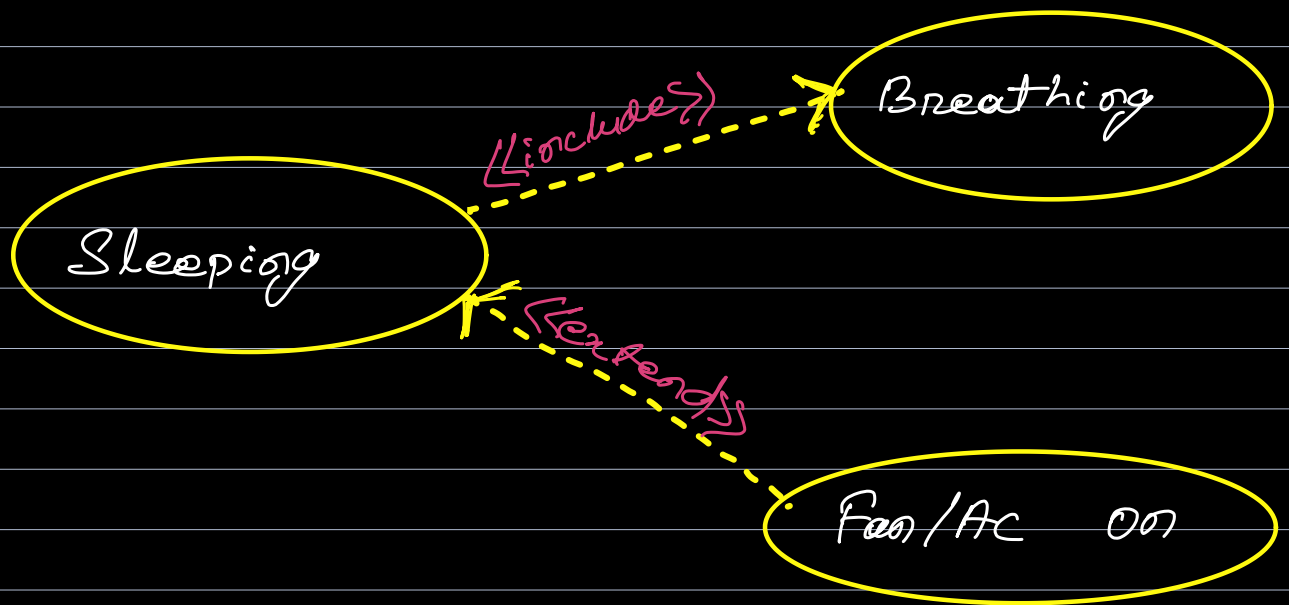
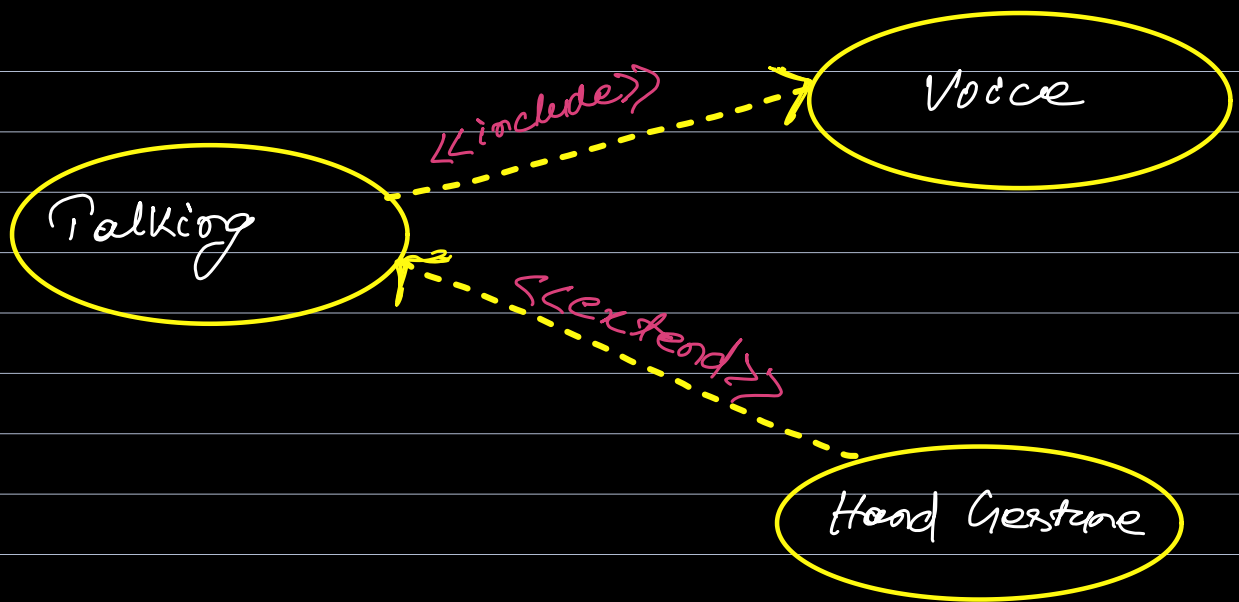
② Optional step

③ Representation :- 



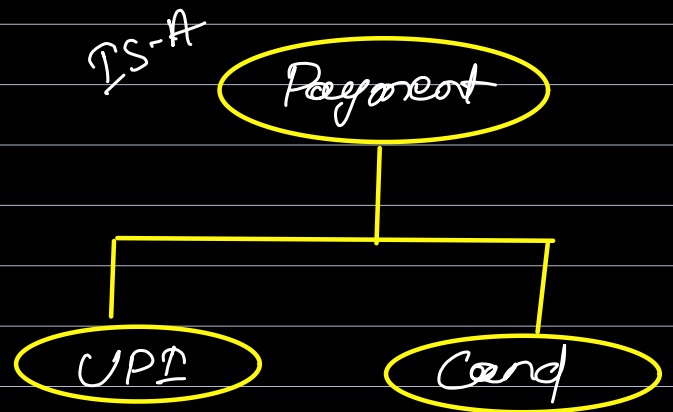
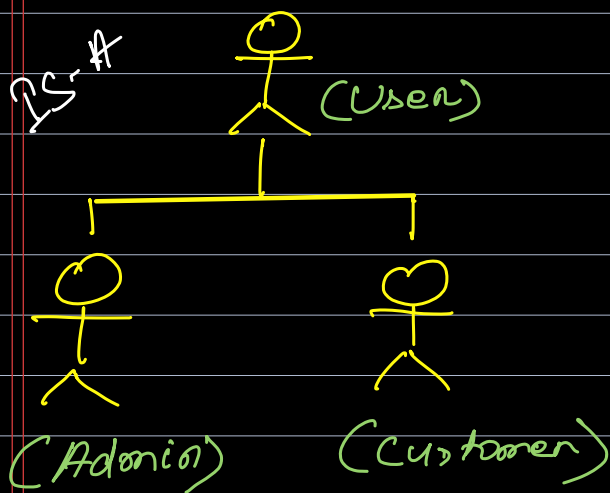
Ex:-





Generalization

- ① Similar to inheritance concept in OOP
- ② Inheritance between Actors or Usecases.
- ③ Representation :-



UML CLASS DIAGRAM

① Definition

② How to represent

- class
- attributes
- methods
- abstract class
- interface
- enum

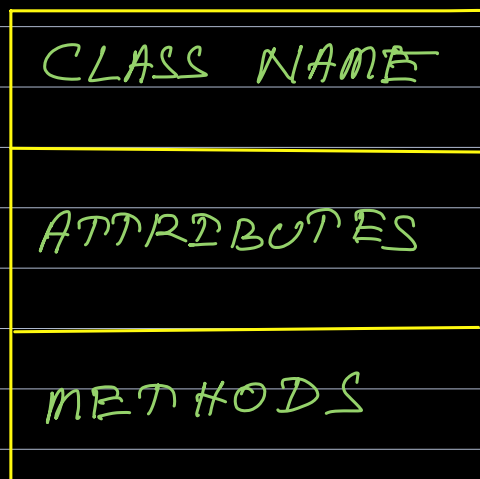
③ Relationships

① Definition

→ Structural UML diagram.

→ It provides a static view of an OOP system, it showcase classes, attributes, methods, and relationship between objects.

② How to represent class, attributes, methods



EXAMPLE :

```
public class Employee {  
    private String name;  
    private int salary;  
    public String address;  
  
    public Employee(String name) {  
        this.name = name;  
    }  
  
    public void print() {  
        System.out.println(this.name);  
    }  
  
    int getSalary() {  
        return this.salary;  
    }  
}
```

How to represent access modifiers?

Access modifiers	Sign
public	+
default	~
private	-
protected	#

Attribute

[access
modifiers] [name of
attribute] • [Data
type]

Ex-1

```
private String name;  
- name : String
```

Ex-2

```
public int age;  
+ age : int
```

Methods & Constructor

[access
modifiers] [name of
method] (data
type of
params) • [return
type]

Ex-1

```
public String getName(int id) {  
    //  
    //  
    //  
}
```

+ getName(id: int) : String

Solve below example

Employee

- name : String
- salary : int
+ address : String

+ Employee(name: String)

+ print(): void

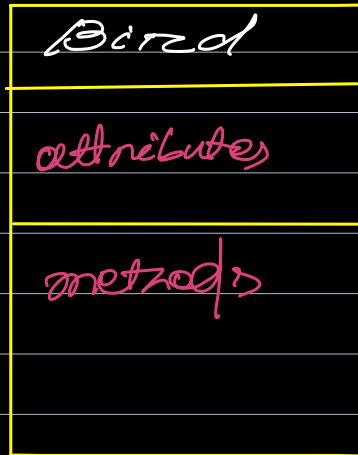
+ getSalary(): int

```
public class Employee {  
    private String name;  
    private int salary;  
    public String address;  
  
    public Employee(String name) {  
        this.name = name;  
    }  
  
    public void print() {  
        System.out.println(this.name);  
    }  
  
    int getSalary() {  
        return this.salary;  
    }  
}
```

③ How to represent abstract class.

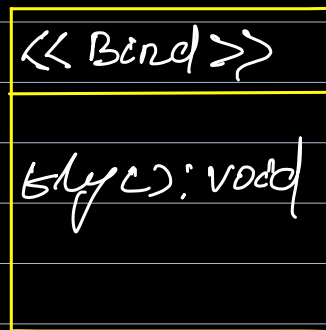
→ class name should be in Italic

Ex1



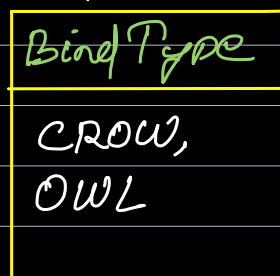
④ How to represent interface?

→ name is in angular bracket
→ no access modifier.



⑤ How to represent enum?

→ Rectangle with 2 parts



⑥ How to represent static attributes/method?

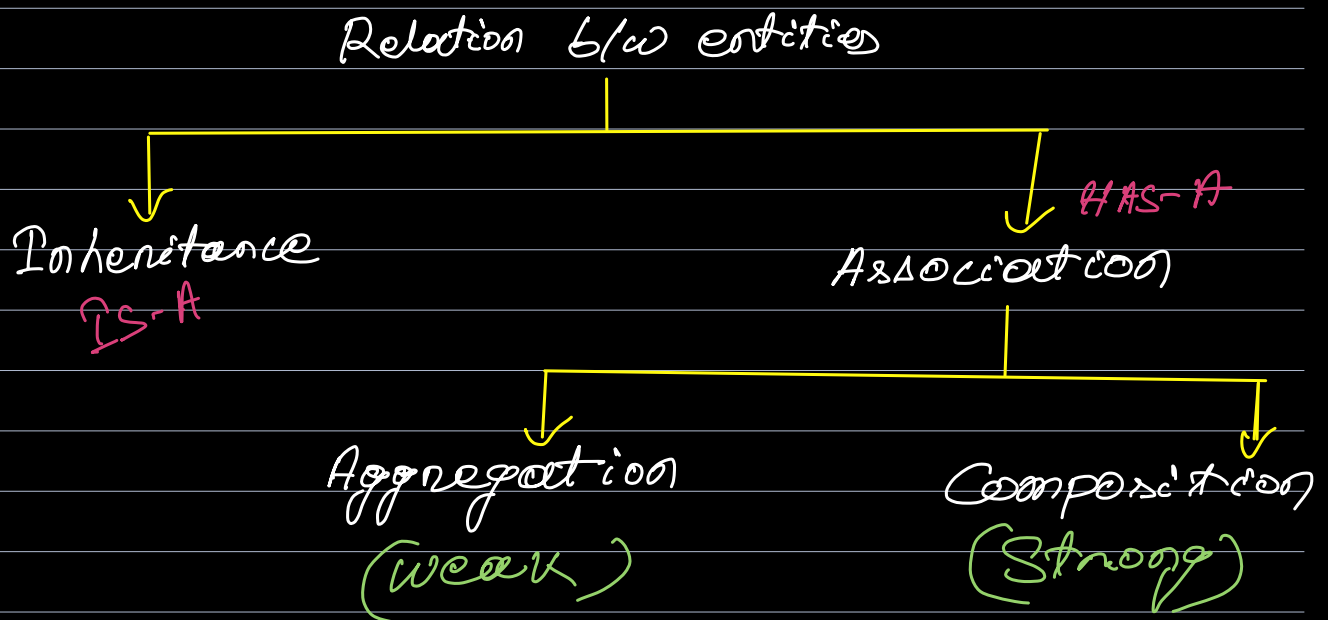
→ Use underscore (_____)

Ex:

private static int age;

— age: int

RELATIONSHIP



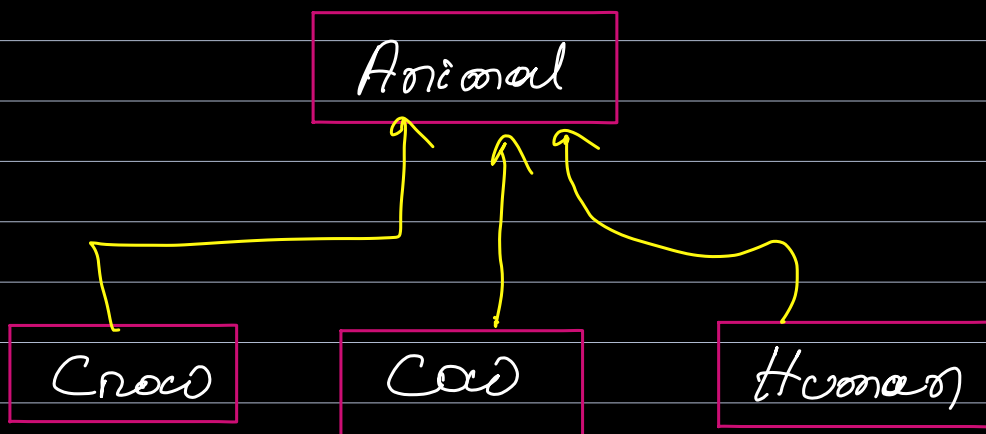
Inheritance

→ It has **IS-A** relationship.

→ Represent :- 



Ex:-



Aggregation

- Aggregation is a "HAS-A" relationship where one class contains another class, but both can exist independently.
- This is also known as weak relationship.
- Representation : 

Aggregate → Collect / combine /

many things combined into one group.

Examples

- * Department HAS-A Employees
Employee can exist without a department



Code

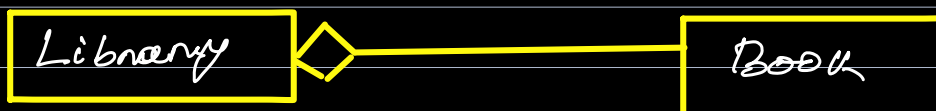
```
class Employee {  
    String name;  
}
```

```
class Department {  
    List<Employee> emp;  
}
```



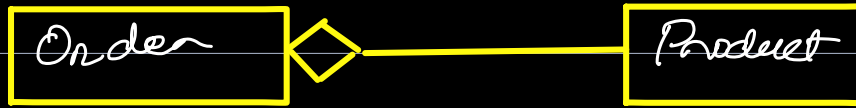
Example-2

- * Library HAS-A Books
Books exist even if library is removed



Example-3

Order HAS-A Products
products exist independently of orders,



Composition

- Composition is a HAS-A relationship where the child object can't exist without the parent.
- This is also known as Strong relationship.
- Representation :- 

Compose → Create / build
Create something meaningful.

Ex-1) Musician composes a song

Example

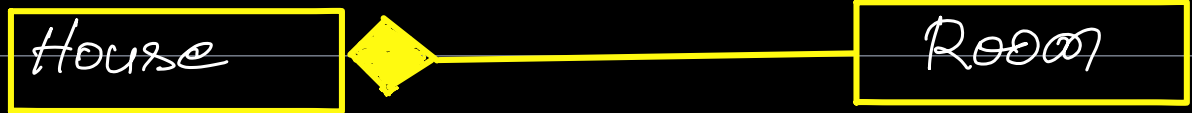
House HAS-A Rooms

If house is destroyed → room doesn't exist

code

```
class Room {  
}
```

```
class House {  
    Room r = new Room();  
}
```



Example-2

User HAS-A Address



Example-3

Car HAS-A Engine



Aggregation		Composition	
*	Weak HAS-A		Strong HAS-A
*	Independent		Dependent
*			